

NAME: COLLINS AMADI
STUDENT ID: FWSD2511357
STUDENT EMAIL: fwsd2511357@students.icdfa.edu.ng
COURSE CODE: SBT-DF203
LAB NAME: Firewall Traffic Control and Forensic Verification
DATE: 17/09/2026
INSTRUCTOR: AMINU IDRIS

Important Pre-Lab Instruction

Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.

Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	Lab 6
Lab Title	Firewall Traffic Control and Forensic Verification
Delivery Block	2/3 of 3
Scheduled Dates	12-18 September 2026
Estimated Duration	4-5 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	Baseline and blocked HTTP PCAPNG files; iptables ruleset exports

1. Source Materials and Slide Alignment

- Firewall: network-based and host-based firewalls, iptables chains and rules, ACCEPT/DROP actions, UFW relationship, NFQUEUE concept and HTTP blocking.
- The slide practical lists rules, adds/removes an INPUT rule, uses a local Apache page and observes allowed versus blocked hosts.
- The homework requires traffic generated by curl from a different machine and forensic observation of the result.

Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded SBT-DF203 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

You are validating whether a Linux host-based firewall correctly blocks HTTP traffic from one authorized lab client while allowing another. You will capture baseline traffic, apply a narrowly scoped iptables rule, capture the blocked connection behaviour, prove the rule hit using counters, remove the rule and document the evidential differences.

3. Learning Outcomes

- Explain the purpose of host-based and network-based firewalls.
- Interpret INPUT, OUTPUT and FORWARD chains.
- Create, verify and remove a narrowly scoped iptables rule.
- Capture allowed and blocked HTTP connection attempts.
- Correlate packet evidence with firewall rule counters and application behaviour.
- Explain DROP versus REJECT and identify retransmission/timeout evidence.
- Restore the original ruleset safely.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.
- Before making changes, export the existing ruleset. Apply only a source-specific rule for the assigned lab IP. Never flush all firewall rules on a remotely administered or production system.

5. Required Tools and Environment

Item	Recommended Tool	Purpose
Server VM	Kali/Ubuntu with Apache	Protected host.
Client VM	Linux or Windows on host-only network	Generate HTTP requests.
Firewall tools	iptables, iptables-save	Apply and preserve rules.
Capture tools	tshark/Wireshark	Capture baseline and blocked traffic.
Client tools	curl, browser	Test access.
Optional extension	NetfilterQueue	Instructor-led packet interception demonstration.

6. Lab Folder Structure and Evidence Preparation

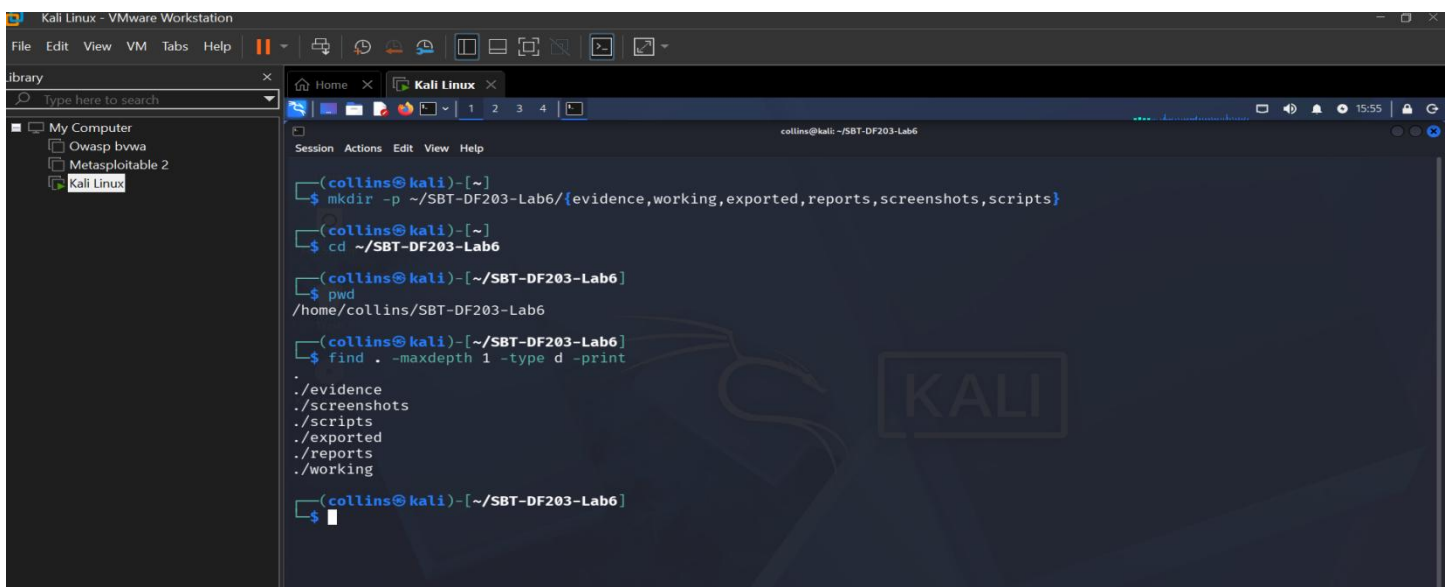
```
mkdir -p ~/SBT-DF203-Lab6/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab6
pwd
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
sudo apt install -y apache2 curl iptables tshark wireshark
sudo systemctl enable --now apache2
printf '<!DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>\n' | sudo tee /var/www/html/firewall_lab.html
sudo iptables-save | tee reports/iptables_before.rules
sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
```

Screenshot required: Original firewall ruleset, rule listing, hash and the training webpage.

ANSWER:



```
Kali Linux - VMware Workstation
File Edit View VM Tabs Help
library
Type here to search
My Computer
Owasp bvwa
Metasploitable 2
Kali Linux
collins@kali: ~/SBT-DF203-Lab6
$ mkdir -p ~/SBT-DF203-Lab6/{evidence,working,exported,reports,screenshots,scripts}
$ cd ~/SBT-DF203-Lab6
$ pwd
/home/collins/SBT-DF203-Lab6
$ find . -maxdepth 1 -type d -print
.
./evidence
./screenshots
./scripts
./exported
./reports
./working
$
```

```

Kali Linux - VMware Workstation
File Edit View VM Tabs Help

Library
Type here to search
My Computer
Owasp bwwa
Metasploitable 2
Kali Linux

(collins@kali)-[~/SBT-DF203-Lab6]
$ sudo systemctl enable --now apache2

(collins@kali)-[~/SBT-DF203-Lab6]
$ printf '<!DOCTYPE html>\n<html>\n<body>\n<h1>ICDFA Network Forensics Firewall Lab</h1>\n<p>Server: YOUR SERVER NAME</p>\n</body>\n</html>\n' | sudo tee /var/www/html/firewall_lab.html
<!DOCTYPE html>
<html>\n<body>\n<h1>ICDFA Network Forensics Firewall Lab</h1>\n<p>Server: YOUR SERVER NAME</p>\n</body>\n</html>

(collins@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables-save | tee reports/iptables_before.rules

(collins@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num pkts bytes target prot opt in out source destination
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num pkts bytes target prot opt in out source destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num pkts bytes target prot opt in out source destination

(collins@kali)-[~/SBT-DF203-Lab6]
$ sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
e3b0c44298fc149afb4c8996fb92427ae41e46496934ca495991b7852b855 reports/iptables_before.rules

(collins@kali)-[~/SBT-DF203-Lab6]
$

```

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab6-COLLINS KELECHI AMADI
Trainee name	COLLINS AMADI
Date and time started	17-09-2026 03:07:45
Evidence file name(s)	SBT-DF203LAB6_FIREWAL_TRAFFIC_CONTROL_AND_FORENSIC_VERIFICATION
Source or generation method	COLLINS@KALI
Original SHA-256	sha256sum reports/iptables_before.rules
Working-copy SHA-256	e3b0c44298fc149afb4c8996fb92427ae41e46496934ca495991b7852b855 reports/iptables_before.rules
Analysis workstation/VM	KALI LINUX COMMAND
Notes on any changes	NILL

8. Part A - Record the Lab Network and Baseline Access

```

# On the server
ip -br address | tee reports/server_interfaces.txt
ip route | tee reports/server_routes.txt
sudo ss -lntp | grep ':80' | tee reports/apache_listener.txt

# On the client, replace SERVER_IP
curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html 2>&1 | tee baseline_curl.txt

```

Record the server IP, allowed client IP, blocked client IP, interface and default gateway. Take a snapshot before changing the firewall.

```

Kali Linux - VMware Workstation
File Edit View VM Tabs Help

Library
Type here to search
My Computer
Owasp bwwa
Metasploitable 2
Kali Linux

(collins@kali)-[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 5 http://192.168.188.130/firewall_lab.html 2>&1 | tee baseline_curl.txt
* Trying 192.168.188.130...
* Established connection to 192.168.188.130 (192.168.188.130 port 80) from 192.168.188.130 port 53512
% Total % Received % Xferd Average Speed Time Time Time Current
 0     0    0     0    0     0      0      0      0  0* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.188.130
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 17 Sep 2026 15:05:50 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 17 Sep 2026 14:59:27 GMT
< ETag: "77-65baf07c1ee27"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
[119 bytes data]
100  119 100  119  0  24947  0
* Connection #0 to host 192.168.188.130:80 left intact
<!DOCTYPE html>
<html>\n<body>\n<h1>ICDFA Network Forensics Firewall Lab</h1>\n<p>Server: YOUR SERVER NAME</p>\n</body>\n</html>

(collins@kali)-[~/SBT-DF203-Lab6]
$

```

ANSWER:

9. Part B - Capture the Allowed HTTP Baseline

```
# On server, replace IFACE and CLIENT_IP
sudo tshark -i IFACE -f 'host CLIENT_IP and tcp port 80' -a duration:30 -w evidence/http_allowed.pcapng &
sleep 3
# Run curl from client during this capture
wait
sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
```

ANSWER:

```
(collins@kali)-[~/SBT-DF203-Lab6]
$ sudo tshark -i IFACE -f '192.168.188.130 and tcp port 80' -a duration:30 -w evidence/http_allowed.pcapng 6
[1] 11884

(collins@kali)-[~/SBT-DF203-Lab6]
$ Running as user "root" and group "root". This could be dangerous.
Capturing on 'IFACE'
tshark: There is no device named "IFACE".
(No such device exists)

[1] + exit 1      sudo tshark -i IFACE -f '192.168.188.130 and tcp port 80' -a duration:30 -w
(collins@kali)-[~/SBT-DF203-Lab6]
$ sleep 3

(collins@kali)-[~/SBT-DF203-Lab6]
$ wait

(collins@kali)-[~/SBT-DF203-Lab6]
$ sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
sha256sum: evidence/http_allowed.pcapng: No such file or directory

(collins@kali)-[~/SBT-DF203-Lab6]
$
```

10. Part C - Apply and Verify the Blocking Rule

Use the exact source IP of the assigned blocked client. Insert the rule near the top of INPUT so it is evaluated before a broad ACCEPT rule.

```
BLOCKED_CLIENT_IP=192.168.56.20
sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
printf 'Rule verified present.\n' | tee reports/rule_verification.txt
```

ANSWER:

```
(collins@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -I INPUT 1 -s "$BLOCKED 192.168.56.20" -p tcp --dport 80 -j DROP

(collins@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
Chain INPUT (policy ACCEPT 1 packets, 76 bytes)
num  pkts bytes target    prot opt in     out     source            destination
1      0    0 DROP      tcp  --  *      *        192.168.56.20      0.0.0.0/0          tcp dpt:80

(collins@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -C INPUT -s "$BLOCKED 192.168.56.20" -p tcp --dport 80 -j DROP

(collins@kali)-[~/SBT-DF203-Lab6]
$ printf 'Rule verified present.\n' | tee reports/rule_verification.txt
Rule verified present.

(collins@kali)-[~/SBT-DF203-Lab6]
$
```

11. Part D - Capture Blocked Traffic and Rule Counters

```
# On server
sudo tshark -i IFACE -f 'host 192.168.56.20 and tcp port 80' -a duration:35 -w evidence/http_blocked.pcapng &
sleep 3
# From blocked client during capture:
# curl -v --connect-timeout 10 http://SERVER_IP/firewall_lab.html
wait
```

```
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
```

ANSWER:

```
(collins@kali)~[/SBT-DF203-Lab6]
$ curl -v --connect-timeout 10 http://192.168.188.130/firewall_lab.html
* Trying 192.168.188.130:80...
* Established connection to 192.168.188.130 (192.168.188.130 port 80) from 192.168.188.130 port 43018
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.188.130
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 17 Sep 2026 15:18:51 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 17 Sep 2026 14:59:27 GMT
< ETag: "77-65baf07c1ee27"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>
* Connection #0 to host 192.168.188.130:80 left intact

(collins@kali)~[/SBT-DF203-Lab6]
$ wait

(collins@kali)~[/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
Chain INPUT (policy ACCEPT 22 packets, 1826 bytes)
num  pkts bytes target    prot opt in     out     source            destination
1      0    0 DROP      tcp  --  *      *        192.168.56.20      0.0.0.0/0          tcp dpt:80

(collins@kali)~[/SBT-DF203-Lab6]
$ sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
sha256sum: evidence/http_blocked.pcapng: No such file or directory
```

12. Part E - Compare Allowed and Blocked Captures

```
for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do
  echo "=====$PCAP===="
  tshark -r "$PCAP" -Y 'tcp.flags.syn==1 || http.request || http.response || tcp.analysis.retransmission' \
    -T fields -e frame.number -e frame.time_relative -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
    -e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e http.response.code
done | tee reports/allowed_vs_blocked.tsv
```

Indicator	Allowed Capture	Blocked Capture
Client SYN visible?	Yes client initiate TCP HANSHAKE	No client attempt SYN
Server SYN-ACK visible?	Yes, server responds	No, firewall drop packet
Handshake completed?	Yes, SYN-SYN-ACK	No, Handshake Falls
HTTP GET visible?	Yes client send request	No, request never sent
HTTP response visible?	Yes, server replies up to 119 ok	No, no response
Retransmissions/timeout	Yes, but normal	YYes, repeated SYNs Timeout
iptables counter change	No, drop counter increament	Yes, drop rule counters increase
curl result	Success page retrieved	Failure, connection timed out

13. Part F - Remove the Rule and Restore Access

```
BLOCKED_CLIENT_IP=192.168.56.20
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP || echo 'Rule successfully removed'
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
# Confirm access from the previously blocked client
# curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html
```

ANSWER:

The screenshot shows a Kali Linux terminal window with the following output:

```
Chain INPUT (policy ACCEPT 26 packets, 2382 bytes)
num  pkts bytes target    prot opt in     out     source                 destination

(collins@kali)~/SBT-DF203-Lab6
$ curl -v --connect-timeout 5 http://192.168.188.130/firewall_lab.html
* Trying 192.168.188.130:80 ...
* Established connection to 192.168.188.130 (192.168.188.130 port 80) from 192.168.188.130 port 51370
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.188.130
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 17 Sep 2026 15:30:08 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 17 Sep 2026 14:59:27 GMT
< ETag: "77-65baf07c1ee27"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>
* Connection #0 to host 192.168.188.130:80 left intact

(collins@kali)~/SBT-DF203-Lab6
$
```

14. Part G - Optional NFQUEUE Observation

Instructor-Led Extension

The slide introduces NFQUEUE and Python packet callbacks. Complete this only in the prepared lab VM because package names and Python bindings vary. The core assessed task is iptables rule evidence and PCAP verification.

- Explain that NFQUEUE passes matching packets to a userspace program.
- Identify where a callback can accept, modify or drop the packet.
- Document the rule, queue number, process and cleanup command.
- Never leave an NFQUEUE rule active without a listening process because traffic may stall.

15. Forensic Interpretation Questions

1. Why does DROP commonly cause SYN retransmissions and a timeout?

ANSWER:

The client receives no response (neither a SYN-ACK nor a RST), its TCP stack assumes the packet was lost in transit and triggers automated **SYN retransmissions** based on its retransmission timer until it eventually reaches its maximum retry limit and **times out**.

2. How would a REJECT rule differ in the packet capture and client output?

ANSWER:

- **Packet Capture:** Instead of silence, the packet capture will show an immediate **RST (Reset) packet** or an **ICMP Destination Unreachable** (e.g., port unreachable) packet sent from the firewall back to the client.

- **Client Output:** Instead of hanging until a timeout, the client application will terminate instantly with an explicit error message such as "**Connection refused**" or "Network unreachable."

3. Why are firewall counters valuable corroborating evidence?

ANSWER:

Firewall counters provide a **tamper-evident log of rule execution** directly from the kernel space. They allow an analyst to correlate the exact volume and timing of blocked network traffic against packet captures, proving that a specific rule was actively intercepting and dropping the packets in question rather than the traffic being lost elsewhere on the network

4. What evidence would show that the web server was down rather than firewall-blocked?

ANSWER:

If the web server is down but unblocked, a packet capture will show that the **TCP handshake completes successfully** (the firewall allows the SYN and SYN-ACK through), but the connection is immediately closed, or the server returns an **HTTP 502/503 Gateway Error** or a RST packet originating directly from the web server's application port rather than the firewall

5. What is the risk of deleting a rule by line number after other rules have changed?

ANSWER:

iptables line numbers are **dynamic and shift automatically** whenever rules are added or removed above them. If you delete a rule by a hardcoded line number after the chain has changed, you risk **deleting the wrong rule entirely**, which can accidentally expose the system to malicious traffic or accidentally block legitimate production services